
Beyond Classical Search (Non-determinism & Partial Observability) + Automated Planning

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Some figures are obtained from the [AIMA Book](#)
- Some automated planning slides from [Nau](#) and [Lima](#).

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Rough overview of what we covered

- **Classical State-Space Search (path planning)**: given a known transition model (explicit graph or successor function) and a goal, computationally search for a (least-cost) path.
 - Deterministic, fully observable environment where the transition mechanics are known.
 - Search algorithms that find a solution: BFS, DFS, UCS, A*, etc.
 - **Important assumption**: action mechanics are given “procedurally” (we did not explicitly model action preconditions/effects).
- **Local Search and Constraint Satisfaction (optimization)**: given an environment (variables) and conditions (constraints), computationally search for a state that satisfies the conditions.

Navigation & Searching

Image from: [link](#)

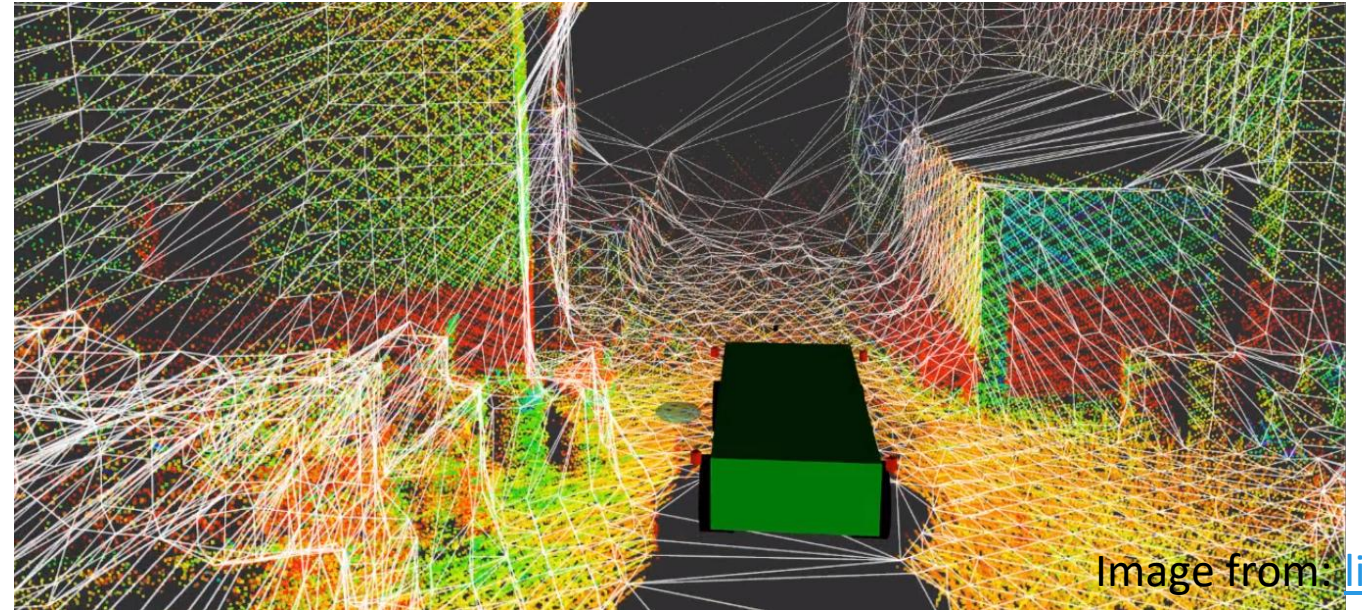
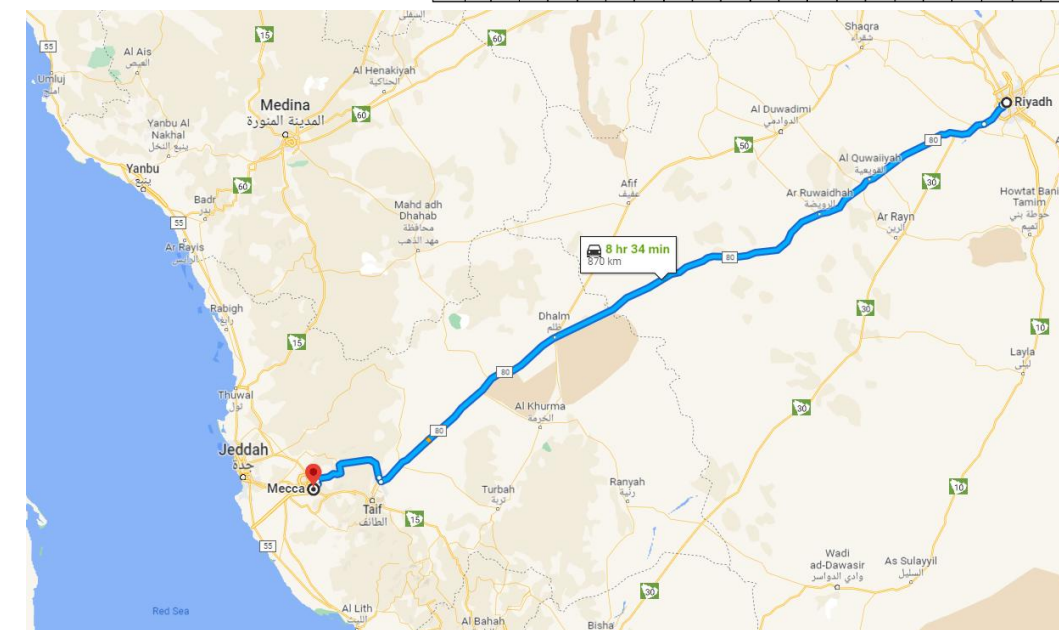
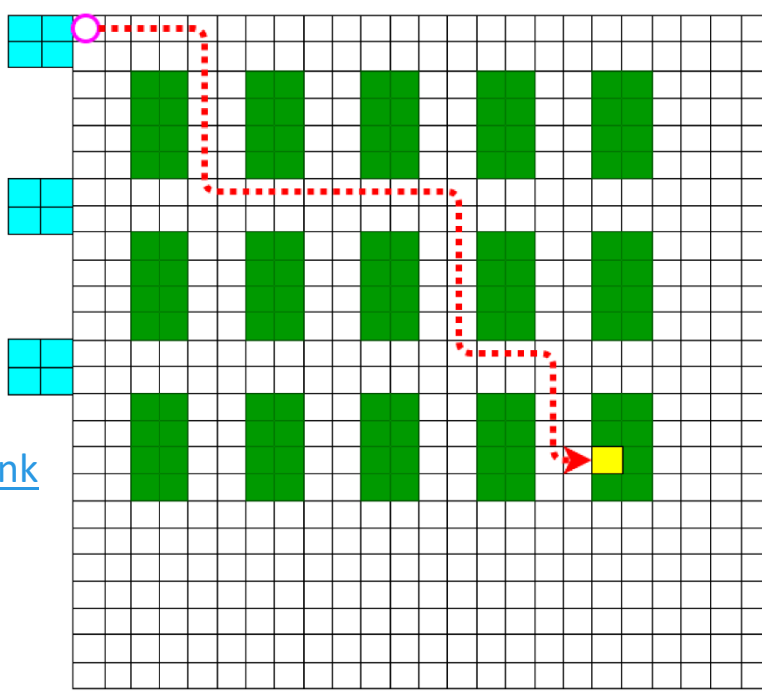


Image from: [link](#)

Scheduling/Optimizing

- Finding optimal configuration of a system.
- Job scheduling
- Circuit design
- Cargo packing
- Flight scheduling
- Registrar scheduling

Course Offering

Select Term

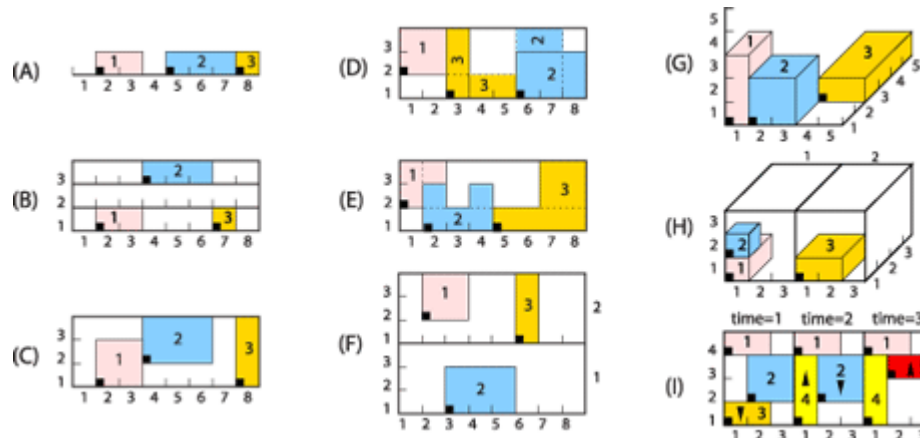
Term 222

Select Department

Accounting & Finance

Course-Sec	Activity	CRN	Course Name	Instructor	Day	Time	Location	Status	Waitlist
ACCT110-01	LEC	21684	Introduction to Financial Accounting	ABDULLAH ALHARBI	MW	1230-1345	24-174	Closed	Open
ACCT110-02	LEC	21083	Introduction to Financial Accounting	ABDULLAH ALHARBI	MW	1400-1515	24-174	Open	Closed
ACCT210-01	LEC	21087	Introduction to Managerial Accounting	MIRGHANI AHMED	UT	1100-1215	24-174	Closed	Open
ACCT210-04	LEC	21090	Introduction to Managerial Accounting	WALID CHEFFI	MW	1230-1345	24-244	Open	Closed
ACCT210-05	LEC	21982	Introduction to Managerial Accounting	MIRGHANI AHMED	UT	1400-1515	24-250	Open	Closed

Course-instructor-room scheduling



Packing problems. Image from [link](#)

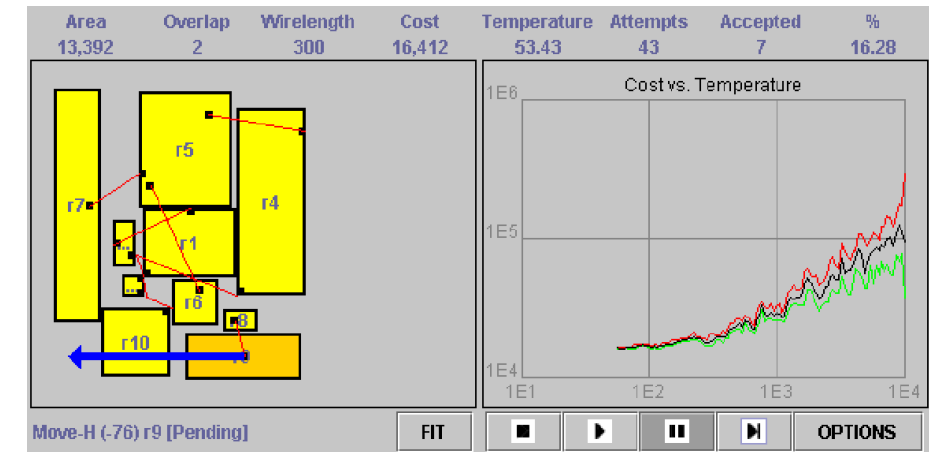


Figure 5 – Simulated Annealing Placement Applet

Circuit design. Image from [link](#)

Beyond Classical Search: Planning in Complex Environments (Overview)

- **Planning/Search in complex environments:**
 - Handle **partial observability** using **belief states** (search over what you know vs what is true).
 - Handle **non-deterministic and probabilistic actions** using **AND/OR search** (plans branch on outcomes).
- **Automated Planning (model-based, domain-independent):**
 - A rich AI field concerned with planning algorithms. Has popular formulation languages and algorithms. Has tournaments where researchers compete in various complex environments.
 - PDDL: represent actions with **preconditions/effects**, plus initial state and goal.
 - Use a **planner** solver to generate a plan.
 - HDDL: represent complex models in a hierarchical fashion with non-primitive tasks that decompose.

Learning Outcomes

- By the end of this lecture, students can:
 - Be aware of other searching and planning approaches.
 - Explain how nondeterminism and partial observability motivate belief-state search and contingent plans. Using AND/OR search and Online search to handle such environments.
 - Describe what automated planning is and distinguish it from classical searchers.
 - Become familiar with planning paradigms (domain-specific vs domain-independent vs HTN/configurable).
 - Identify the role of PDDL in domain-independent planning. Interpret and develop a simple PDDL action schema. Then run a solver to generate a plan (solution).
 - Become familiar with Hierarchical Planning, HDDL, and its uses.

And/Or Search and Online Search

Beyond what we covered in searching

- What we covered in searching is one fundamental approach to planning known environments, but there are other approaches in complex environments.
- What to do when there is partial-observability, probabilistic actions, limited time, etc?
 - Search algorithms can be adapted to handle ***possible-next-states*** (also called **belief states**). We will briefly discuss one algorithm called **And/Or searching**.
 - We can do **online searching** (interleave computation and physical actions). Given current info, we perform an incomplete search for a limited amount of time to select the best estimated state (e.g. use A*). Then perform the action physically, gather new environment info via sensors, and rerun another incomplete search.
 - Can use a logic-based or probabilistic-based model to keep track of environment info. The planner can *consult* the logic/probabilistic model to reason/search about which best action to take. Similar to what we did with Prolog pitgridworld example.
 - Learning an optimum strategy can also be done using Reinforcement Learning approach (discussed later).

Searching in nondeterministic and partially observable environments

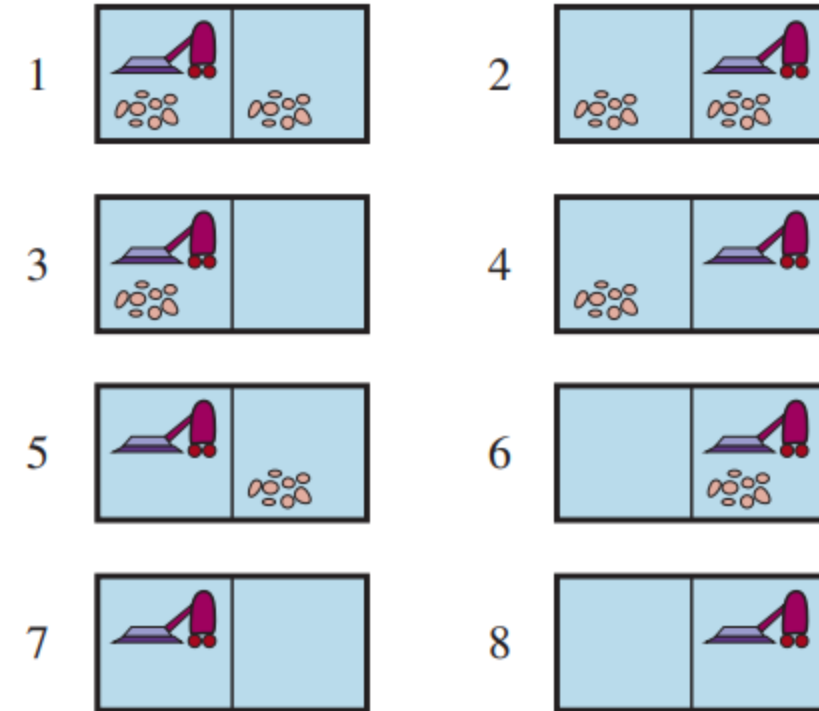
- Nondeterministic: performing the same action in a state may transition to different states.
- Partially observable: the agent does not have full information on the environment (it may be able to gather it).
- **Belief states:** the **set** of physical states that the agent **believes** it is in.
- In nondeterministic and partially observable environments, the agent *computationally* searches through belief states (sets). The solution is a **contingency plan**: a conditional sequence of actions depending on what is actually encountered as it *executes the plan in the real world*.
- There is AND/OR search algorithm that is augmented with belief states.
- A more general framework for handling complex environments is through probabilistic models and rewards. We will see this when we talk about reinforcement learning.

Searching in nondeterministic with example

- In the **erratic vacuum world**, the Suck action works as follows:
 - When applied to a dirty square the action cleans the square and sometimes cleans up dirt in an adjacent square, too.
 - When applied to a clean square the action sometimes deposits dirt on the carpet.
- Nondeterminism:** Result(s, a) returns a set of possible next states (one step of environment branching).

$$\text{RESULTS}(1, \text{Suck}) = \{5, 7\}$$

- We can model the *planning* in this environment as an AND/OR Search Tree. This is the search work done.



AND/OR Search Tree

- OR nodes are **belief state nodes** where agent takes action.
- AND nodes (green circles) splits over all possible outcomes of that action; a solution must handle every AND branch
- **Full observability**: the agent always knows the current world state, so its belief is a singleton ($\{s\}$) in this example.
- So, this environment is fully-observable, non-deterministic.
- A **solution** for an AND–OR search problem is a **subtree** (yellow highlight) of the search tree that translates to conditional plan:

[Suck, if State = 5 then [Right, Suck] else []]

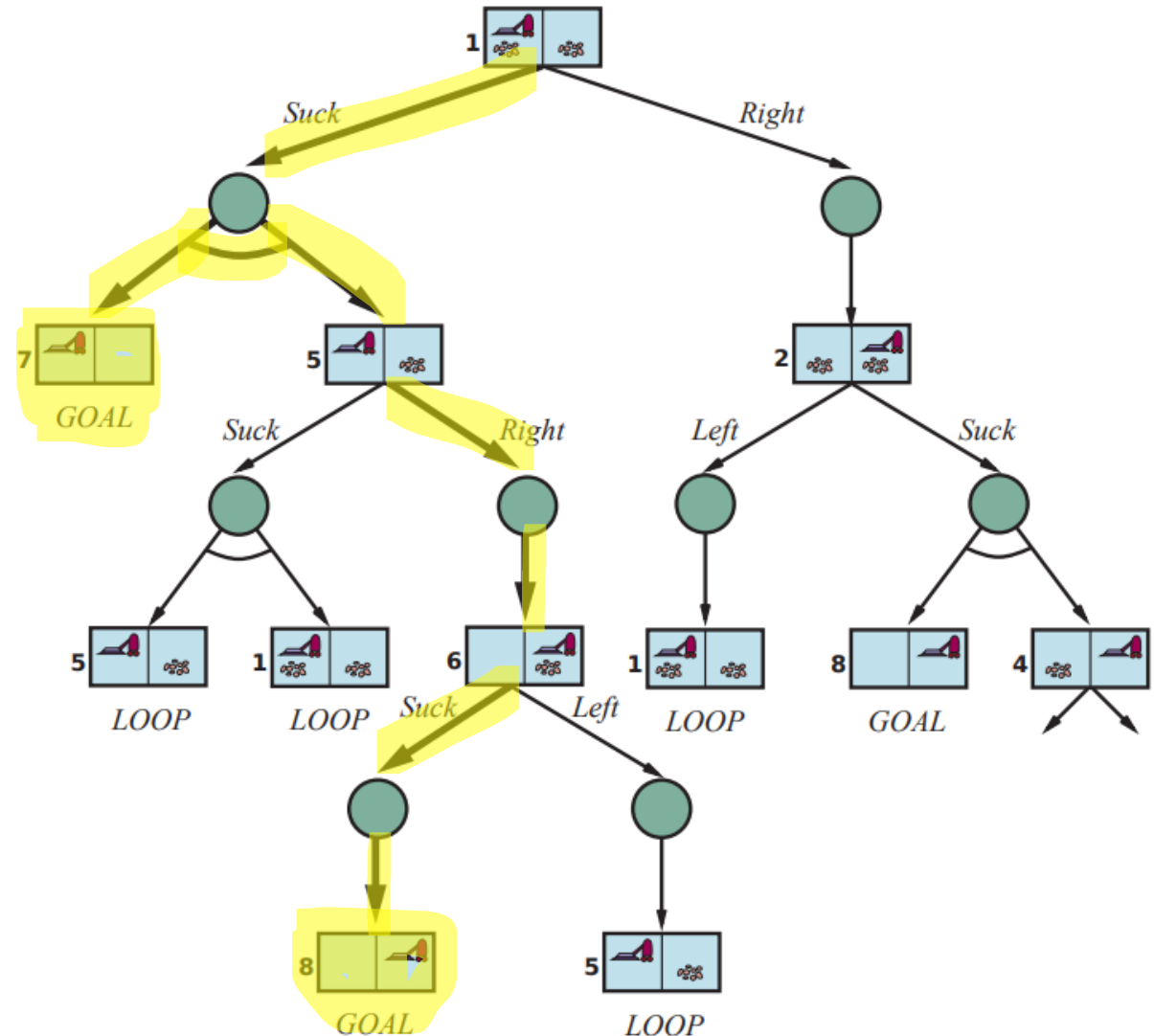


Figure 4.10 The first two levels of the search tree for the erratic vacuum world. State nodes are OR nodes where some action must be chosen. At the AND nodes, shown as circles, every outcome must be handled, as indicated by the arc linking the outgoing branches. The solution found is shown in bold lines.

AND/OR Search

- OR nodes are **belief state** nodes where agent takes action. So, **state** argument here is a **belief state** that can have one state or multiple states.
- AND nodes (**green circles**) splits over all possible outcomes of that action; a solution must handle every AND branch
- Note that **RESULTS(s, a)** returns a set of possible next states.
- *path* is just for detecting cycles.
- More details in [AIMA](#) book chapter 4.4.

```
function AND-OR-SEARCH(problem) returns a conditional plan, or failure
  return OR-SEARCH(problem, problem.INITIAL, [])

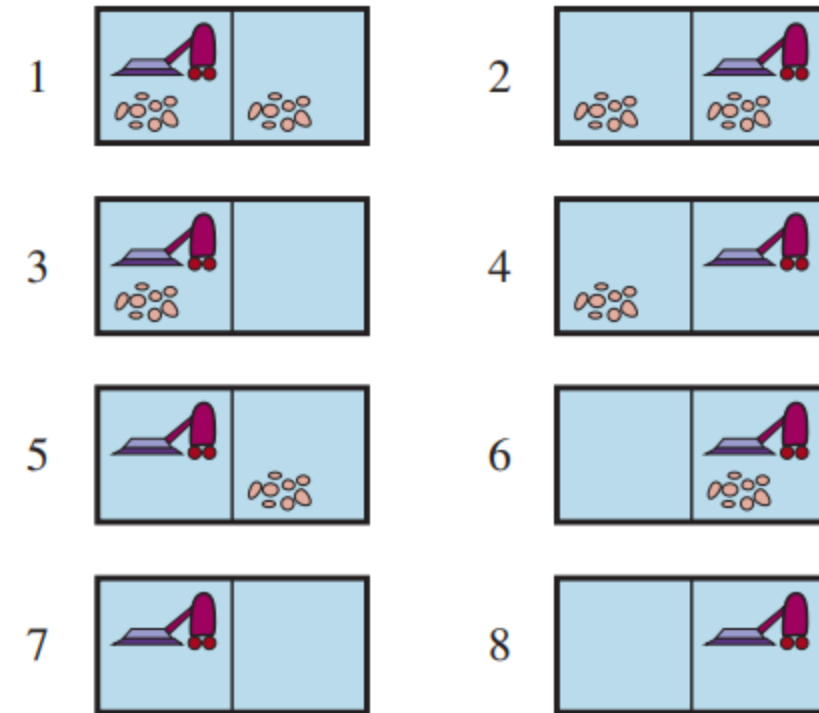
function OR-SEARCH(problem, state, path) returns a conditional plan, or failure
  if problem.IS-GOAL(state) then return the empty plan
  if IS-CYCLE(path) then return failure
  for each action in problem.ACTIONS(state) do
    plan ← AND-SEARCH(problem, RESULTS(state, action), [state] + path)
    if plan ≠ failure then return [action] + plan
  return failure

function AND-SEARCH(problem, states, path) returns a conditional plan, or failure
  for each si in states do
    plani ← OR-SEARCH(problem, si, path)
    if plani = failure then return failure
  return [if s1 then plan1 else if s2 then plan2 else ... if sn-1 then plann-1 else plann]
```

Figure 4.11 An algorithm for searching AND–OR graphs generated by nondeterministic environments. A solution is a conditional plan that considers every nondeterministic outcome and makes a plan for each one.

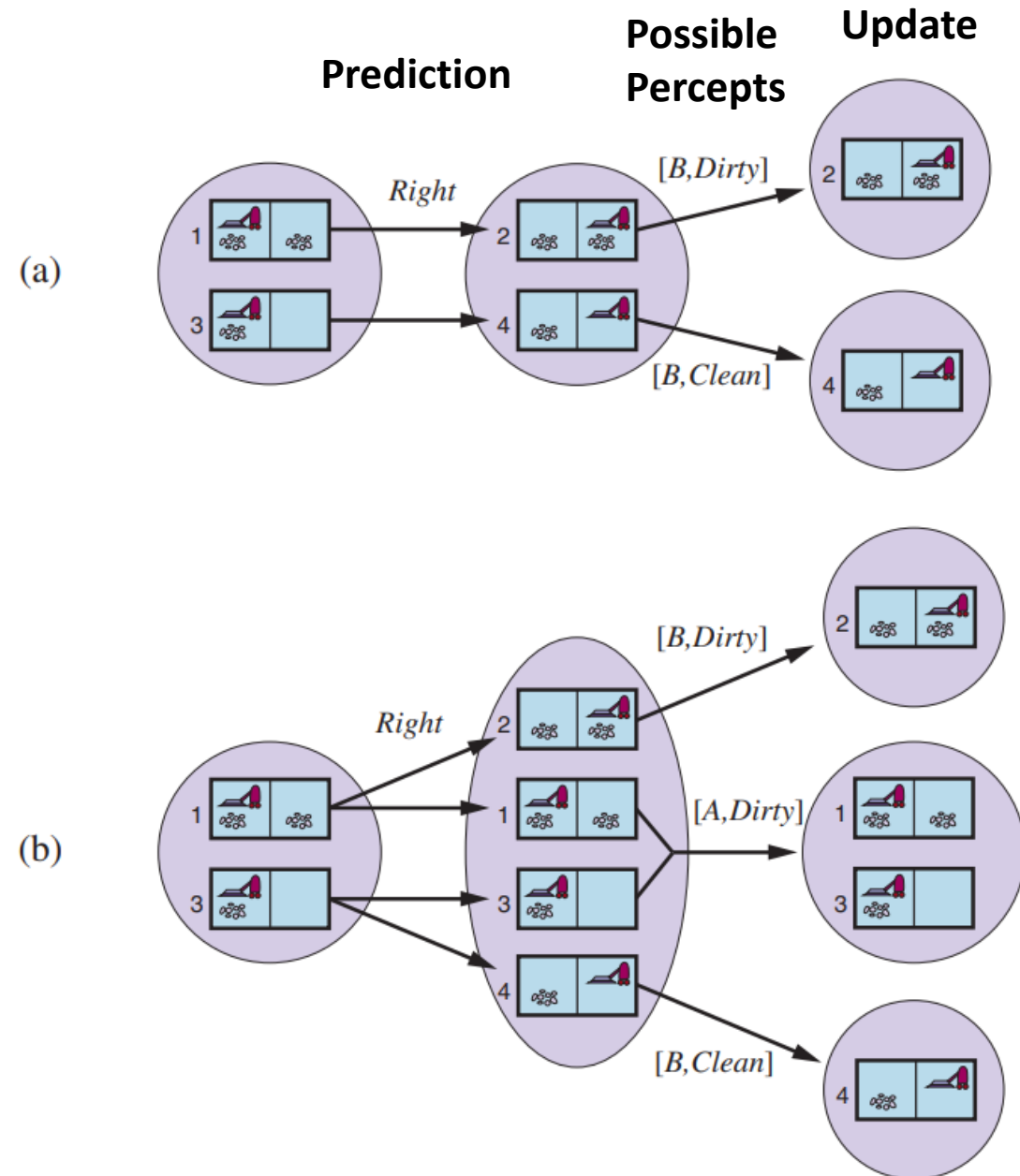
Searching in **partially observable** with example

- Consider a local-sensing vacuum world, in which the agent has:
 - Position sensor that yields the percept A in the left square, and B in the right square.
 - Dirt sensor that yields Dirty when the current square is dirty and Clean when it is clean. So, it cannot observe the status of adjacent grid.
- If agent always starts in left grid, and sense [A, dirt], then it will have belief state = {1, 3}. (it does not know status of B)
- When implementing, we also implement a PERCEPT(s) function for the sensors that returns the percepts sensed in state s.
- Due to partial observability, the transition model RESULTS(b, a) will take in belief state b which can have multiple states, so it requires care in implementation (in three stages):
 - Prediction:** performing action a in belief state b . We predict the resulting states $\hat{b}$ for each $s \in b$.
 - Possible percepts:** for each predicted state in $\hat{b}$, we run PERCEPT(s).
 - Update:** compute for each possible percept, the belief state that would result from the percept.



Partially observable (Deterministic vs nondeterministic)

- (a) In the **deterministic** vacuum world, Right is applied in the initial belief state, resulting in a new **predicted** belief state with two possible physical states; for those states, the **possible percepts** are [B,Dirty] and [B,Clean], leading to two belief states, each of which is a singleton.
- (b) In the **nondeterministic slippery** vacuum world, Right is applied in the initial belief state, giving a new belief state with four physical states; for those states, the possible percepts are [A,Dirty], [B,Dirty], and [A,Clean], leading to three belief states as shown.
- How to search for a solution? We can use AND/OR search.



AND/OR Search

- With partial observability, can still use this same algorithm.
- The main difference is that *state* here is a belief state that can have multiple states. So, when RESULTS(*s*, *a*) processes it, it does the three stages to generate a *list* of belief states.
- *states* in the AND-SEARCH argument now is a list of belief states that each may contain more than one state.
- So we just need to implement RESULTS correctly to do the three stages.
- More details in [AIMA](#) book chapter 4.4.

```
function AND-OR-SEARCH(problem) returns a conditional plan, or failure  
  return OR-SEARCH(problem, problem.INITIAL, [])
```

```
function OR-SEARCH(problem, state, path) returns a conditional plan, or failure  
  if problem.IS-GOAL(state) then return the empty plan  
  if IS-CYCLE(path) then return failure  
  for each action in problem.ACTIONS(state) do  
    plan ← AND-SEARCH(problem, RESULTS(state, action), [state] + path)  
    if plan ≠ failure then return [action] + plan  
  return failure
```

```
function AND-SEARCH(problem, states, path) returns a conditional plan, or failure  
  for each si in states do  
    plani ← OR-SEARCH(problem, si, path)  
    if plani = failure then return failure  
  return [if s1 then plan1 else if s2 then plan2 else ... if sn-1 then plann-1 else plann]
```

Figure 4.11 An algorithm for searching AND–OR graphs generated by nondeterministic environments. A solution is a conditional plan that considers every nondeterministic outcome and makes a plan for each one.

AND/OR Search Partial observability

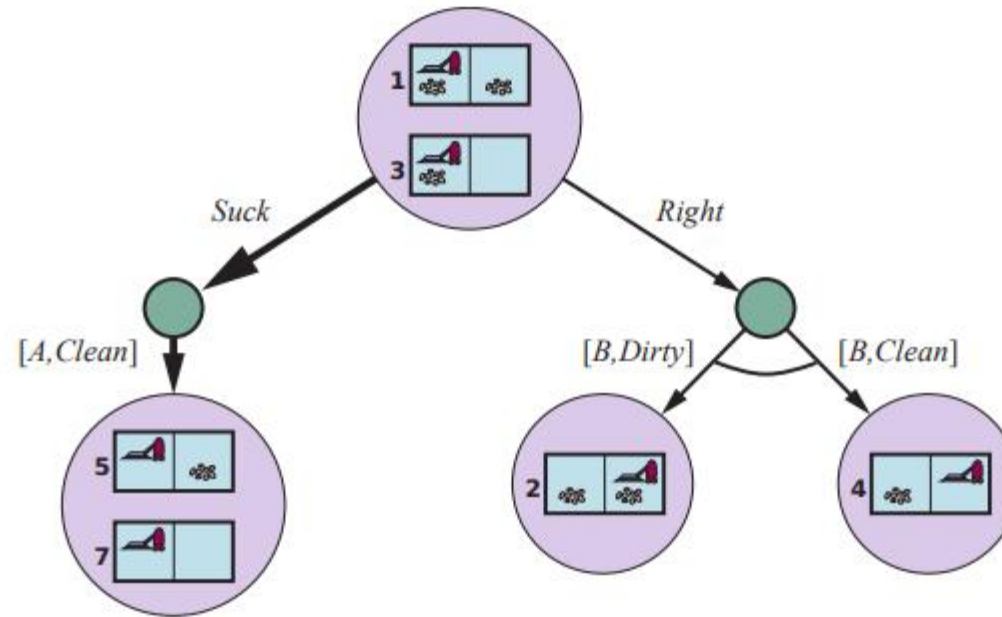


Figure 4.16 The first level of the AND–OR search tree for a problem in the local-sensing vacuum world; *Suck* is the first action in the solution.

- Notice that OR nodes here are belief states that can have multiple states. (as opposed to the earlier example with singletons).
- What is the goal-test? The goal is necessarily achieved if **every** state in the belief state is a goal state.
- The solution is again a conditional plan, but the if-checks are on belief states instead of states.

Online Searching

- Online searching interleaves computation (searching/planning) and acting (physically).
- Suitable in dynamic environment where there is a penalty like time or battery drain.
- Can be used for **mapping** an unknown environment. Agent computes next action, performs action, senses, and updates environment model.
- Can also be used for searching a solution in an unknown environment. Its efficiency depends on the environment; sometimes better to use reinforcement learning approaches.
- Simple online mapping algorithm shown. Can be used for:
 - **Map an unknown environment:** records $result[(s, a)] = s'$ and learns “from here, this action goes there”
 - **Search for a goal while moving:** interleaves “pick an action” with “see where I end up,” like DFS but in the real world.
 - **Backtrack when stuck:** When no untried actions remain at the current state, it uses unbacktracked to return toward a state where it can try something else.
 - Note that this algorithm is simple and used to map an environment. You can inject a more goal-oriented search logic when you take an action (e.g. depth-limited A*)

```
function ONLINE-DFS-AGENT(problem,  $s'$ ) returns an action
     $s$ ,  $a$ , the previous state and action, initially null
    persistent: result, a table mapping  $(s, a)$  to  $s'$ , initially empty
                untried, a table mapping  $s$  to a list of untried actions
                unbacktracked, a table mapping  $s$  to a list of states never backtracked to

    if problem.IS-GOAL( $s'$ ) then return stop
    if  $s'$  is a new state (not in untried) then untried[ $s'$ ]  $\leftarrow$  problem.ACTIONS( $s'$ )
    if  $s$  is not null then
        result[ $s, a$ ]  $\leftarrow$   $s'$ 
        add  $s$  to the front of unbacktracked[ $s'$ ]
    if untried[ $s'$ ] is empty then
        if unbacktracked[ $s'$ ] is empty then return stop
        else  $a \leftarrow$  an action  $b$  such that result[ $s', b$ ] = POP(unbacktracked[ $s'$ ])
    else  $a \leftarrow$  POP(untried[ $s'$ ])
     $s \leftarrow s'$ 
    return  $a$ 
```

Automated Planning

Planning Overview

- **Planning** is a large field in AI. Also called **automated planning**. It encapsulates developing algorithms to plan the sequence of actions in an environment to achieve a goal.
- What we covered in searching are fundamental approaches that are also part of planning, but there are other approaches as well.
- Some problems have a large state-space. Using algorithms we've discussed are infeasible. Finding an optimal solution is also infeasible (opt for a good solution instead).
- Some of what is covered in automated planning textbooks or courses. List taken from Dana Nau webpage. He co-authored a popular [textbook on automated planning](#). Also, overview in [AIMA](#) book chapter 11.
- Such a course would cover what we covered plus:
 - Other searching techniques that use formalized planning languages (PDDL).
 - Hierarchical planning. Can help in large state-spaces.
 - Planning with timing (scheduling).
 - Planning in complex environments.
 - Planning in probabilistic domains.
 - Many more.
- Following slides adapted from [Nau](#) and [Lima](#).

Chap. 1, Introduction

Chap. 2, Representations for Classical Planning

Chap. 3, Complexity of Classical Planning

Chap. 4, State-Space Planning

Chap. 5, Plan-Space Planning

Chap. 6, Planning-Graph Techniques

Chap. 7, Propositional Satisfiability Techniques

Intro. to Part 3

Chap. 9, Heuristics in Planning

Chap. 10, Control Rules in Planning

Chap. 11, Hierarchical Task Network Planning

Chap. 14 sec. 14.3, Planning with Chronicles

Chap. 15, Planning and Resource Scheduling

Chap. 16, Planning on MDPs

Chap. 17, Planning as Model Checking

Overview of Types of Planners

- Domain-independent:
 - In principle, works in any planning domain. By domain, we mean a particular field/specialty.
 - Uses standardized planning domain definition language to define problems (**PDDL**)
 - Less efficient than domain-specific planners, but can express wide range of problems.
 - Examples: we'll see some PDDL-based examples
- Domain-specific:
 - Tuned for a specific planning domain. Exploit specific domain structure and shortcuts.
 - Efficient, but requires custom code development.
 - Won't work well (if at all) in other planning domains
 - Examples: Mars Rover, sheet metal bending machines
- Configurable (midway between domain-independent and domain-specific):
 - Domain-independent planning engine.
 - Includes domain-specific designs on how to *decompose* problems into smaller sub-problems.
 - More efficient than domain-independent, more expressive than domain-specific.
 - Examples: Hierarchical Task Networks (HTN)

Domain-independent Planner – PDDL Overview

- Planning Domain Definition Language (PDDL) is a family of languages for defining a planning problem.
- Learning outcome: Be able to define a small domain and problem in PDDL for a given environment, then run a solver and interpret the plan it returns.
- What PDDL specifies:
 - facts (predicates over objects)
 - action schemas (preconditions and effects)
 - initial state and goal state
- To define a problem in PDDL, you define two files:
 - Domain file: contains logic of the problem via predicates + action schemas
 - Problem file: contains a specific instance of the problem via objects + initial state + goal
- Planner solver takes in your PDDL domain and problem files. Uses search (often heuristic-guided) to find a sequence of actions that achieves the goal.
- Can be used in game AI (see [link](#))

PDDL – Air Transport Cargo Example – Domain file

- Consider a problem where we have planes, cargo, and airports. Planes **fly** between airports; cargo is **loaded** at an airport, **flown** inside a plane, and **unloaded** at another. Below is domain file for this problem.

- We define three types of objects.
- Predicates are how we define **states** in PDDL by describing properties of objects or relationships between them, which are either true or false at any given time.
- Three actions: load, unload, and fly. Notice they have arguments, preconditions, and effects.
- Note we can use this domain file to define multiple problem instance files.

```
1 (define (domain air-cargo)
2   (:requirements :strips :typing)
3   (:types cargo plane airport)
4   (:predicates
5     (at ?x - (either cargo plane) ?a - airport)
6     (in ?c - cargo ?p - plane))
7   (:action load
8     :parameters (?c - cargo ?p - plane ?a - airport)
9     :precondition (and (at ?c ?a) (at ?p ?a))
10    :effect (and (not (at ?c ?a)) (in ?c ?p)))
11  (:action unload
12    :parameters (?c - cargo ?p - plane ?a - airport)
13    :precondition (and (in ?c ?p) (at ?p ?a))
14    :effect (and (at ?c ?a) (not (in ?c ?p))))
15  (:action fly
16    :parameters (?p - plane ?from - airport ?to - airport)
17    :precondition (at ?p ?from)
18    :effect (and (not (at ?p ?from)) (at ?p ?to))))
```

PDDL – Air Transport Cargo Example – Problem file

- Shown problem instance has items (C1, C2), planes (P1, P2) and airports (SFO, JFK).

- We define available typed objects in this instance.

- init defines initial state via preconditions.

- goal defines goal via preconditions.

```
1  (define (problem swap-cargo)
2    (:domain air-cargo)
3    (:objects
4      c1 c2 - cargo
5      p1 p2 - plane
6      sfo jfk - airport)
7    (:init
8      (at c1 sfo)
9      (at c2 jfk)
10     (at p1 sfo)
11     (at p2 jfk))
12   (:goal (and (at c1 jfk) (at c2 sfo))))
```

PDDL – Air Transport Cargo Example – Plan solution

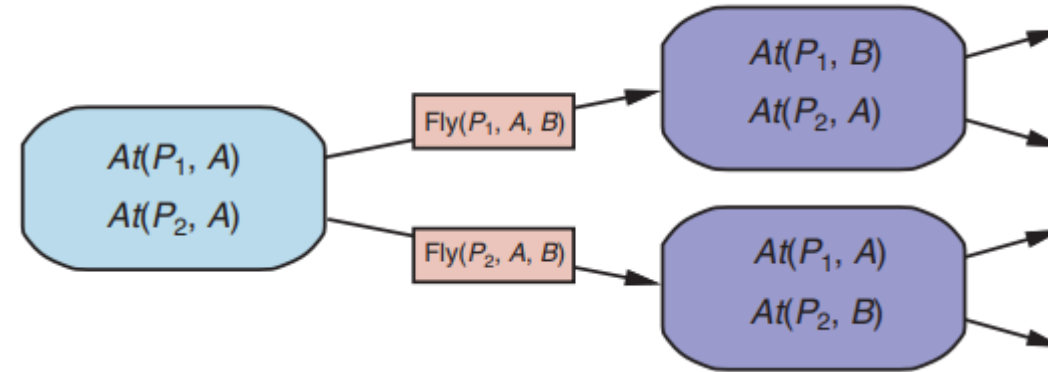
- With these two files, we can use a planner solver at this website <https://editor.planning.domains/>
- Below is the plan solution.

Step	Plan action	Effect	Current State (change in bold)
Init			(at c1 sfo), (at c2 jfk), (at p1 sfo), (at p2 jfk)
1	(fly p2 jfk sfo)	(and (not (at p2 jfk)) (at p2 sfo))	(at c1 sfo), (at c2 jfk), (at p1 sfo), (at p2 sfo)
2	(load c1 p1 sfo)	(and (not (at c1 sfo)) (in c1 p1))	(in c1 p1) , (at c2 jfk), (at p1 sfo), (at p2 sfo)
3	(fly p1 sfo jfk)	(and (not (at p1 sfo)) (at p1 jfk))	(in c1 p1), (at c2 jfk), (at p1 jfk) , (at p2 sfo)
4	(unload c1 p1 jfk)	(and (at c1 jfk) (not (in c1 p1)))	(at c1 jfk) , (at c2 jfk), (at p1 jfk), (at p2 sfo)
5	(fly p2 sfo jfk)	(and (not (at p2 sfo)) (at p2 jfk))	(at c1 jfk), (at c2 jfk), (at p1 jfk), (at p2 jfk)
6	(load c2 p2 jfk)	(and (not (at c2 jfk)) (in c2 p2))	(at c1 jfk), (in c2 p2) , (at p1 sfo), (at p2 jfk)
7	(fly p2 jfk sfo)	(and (not (at p2 jfk)) (at p2 sfo))	(at c1 jfk), (in c2 p2), (at p1 sfo), (at p2 sfo)
8	(unload c2 p2 sfo)	(and (at c2 sfo) (not (in c2 p2)))	(at c1 jfk), (at c2 sfo) , (at p1 sfo), (at p2 sfo)

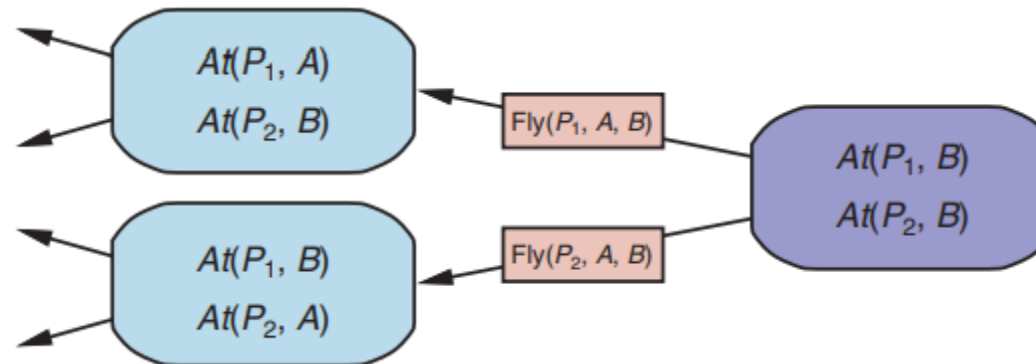
- Notice at the end current facts support the goal: `(:goal (and (at c1 jfk) (at c2 sfo))))`

Finding Solution Plan – Two Approaches

- **Progressive:** forward state-space search. Start from a set of facts (initial state), apply ground actions, get successor facts. Similar to techniques we covered. Can use heuristics with searching.



- **Regressive:** backward relevant-states search. Start from goal, get to initial state. Hard to use heuristics with.



- <https://editor.planning.domains/> supports a number of popular solvers.

- Simple NxN grid where agent starts at initial grid and goal is to get to goal grid.
- Also, some grids are blocked (agent cannot move into them). We use the *free* predicate to denote this.

```
1 (define (domain gridworld-nav)
2   (:requirements :strips :typing)
3   (:types cell)
4   (:predicates
5     (at ?c - cell)
6     (adj ?from - cell ?to - cell)
7     (free ?c - cell))
8   (:action move
9     :parameters (?from - cell ?to - cell)
10    :precondition (and (at ?from) (adj ?from ?to) (free ?to))
11    :effect (and (not (at ?from)) (at ?to))))
```

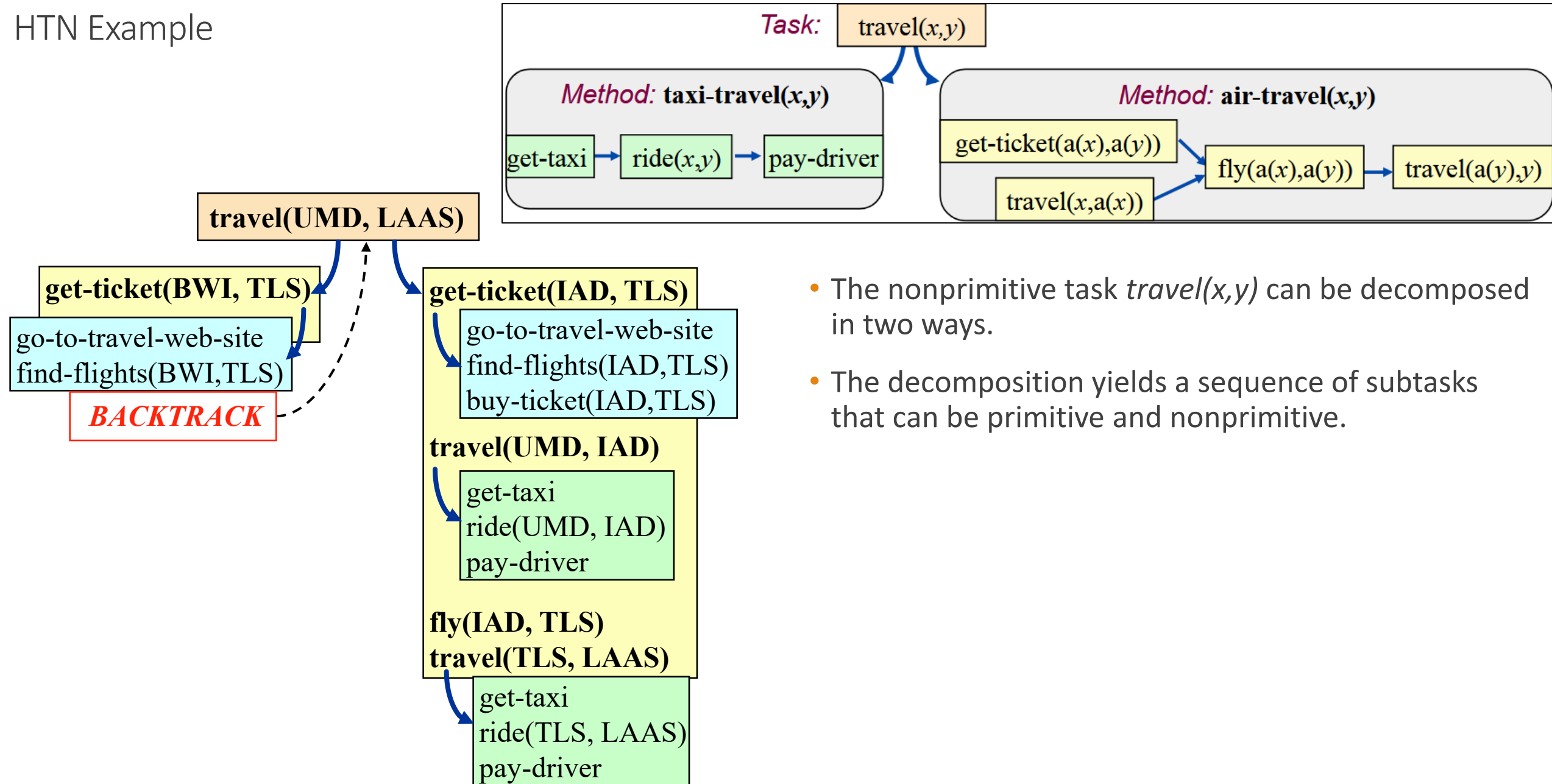
- The instance is 3x3 grid where agent starts in c11 and goal is c33. Grids c21 and c22 are blocked.

```
1 (define (problem nav-3x3)
2   (:domain gridworld-nav)
3   (:objects
4     c11 c12 c13 c21 c22 c23 c31 c32 c33 - cell)
5   (:init
6     (at c11)
7     (free c11) (free c12) (free c13) (free c23) (free c31) (free c32) (free c33)
8     (adj c11 c12) (adj c12 c11)
9     (adj c11 c21) (adj c21 c11)
10    (adj c12 c13) (adj c13 c12)
11    (adj c12 c22) (adj c22 c12)
12    (adj c13 c23) (adj c23 c13)
13    (adj c21 c22) (adj c22 c21)
14    (adj c21 c31) (adj c31 c21)
15    (adj c22 c23) (adj c23 c22)
16    (adj c22 c32) (adj c32 c22)
17    (adj c23 c33) (adj c33 c23)
18    (adj c31 c32) (adj c32 c31)
19    (adj c32 c33) (adj c33 c32))
20  (:goal (and (at c33))))
```

Plan solution is:
(move c11 c12)
(move c12 c13)
(move c13 c23)
(move c23 c33)

Hierarchical Planning Overview

- The main idea is problem decomposition:
 - Tasks (activities) rather than goals.
 - Define methods to decompose tasks into subtasks.
 - Enforce constraints, link sub-solutions, backtrack if necessary.
- Formalized as **Hierarchical Task Networks (HTN)**: **nonprimitive** tasks are decomposed by **methods** (decomposition rules) into subtasks until **primitive** tasks/actions remain.
- Popular hierarchical planners: Noah, Nonlin, O-Plan, SIPE, SIPE-2, SHOP, SHOP2
- HTN is a configurable planner:
 - the general engine still relies on domain-independent PDDL for primitive actions and init/goal facts.
 - HTN adds domain-specific methods that decompose high-level tasks into subtasks
 - So it sits between fully domain-independent classical planning and domain-specific implementations.

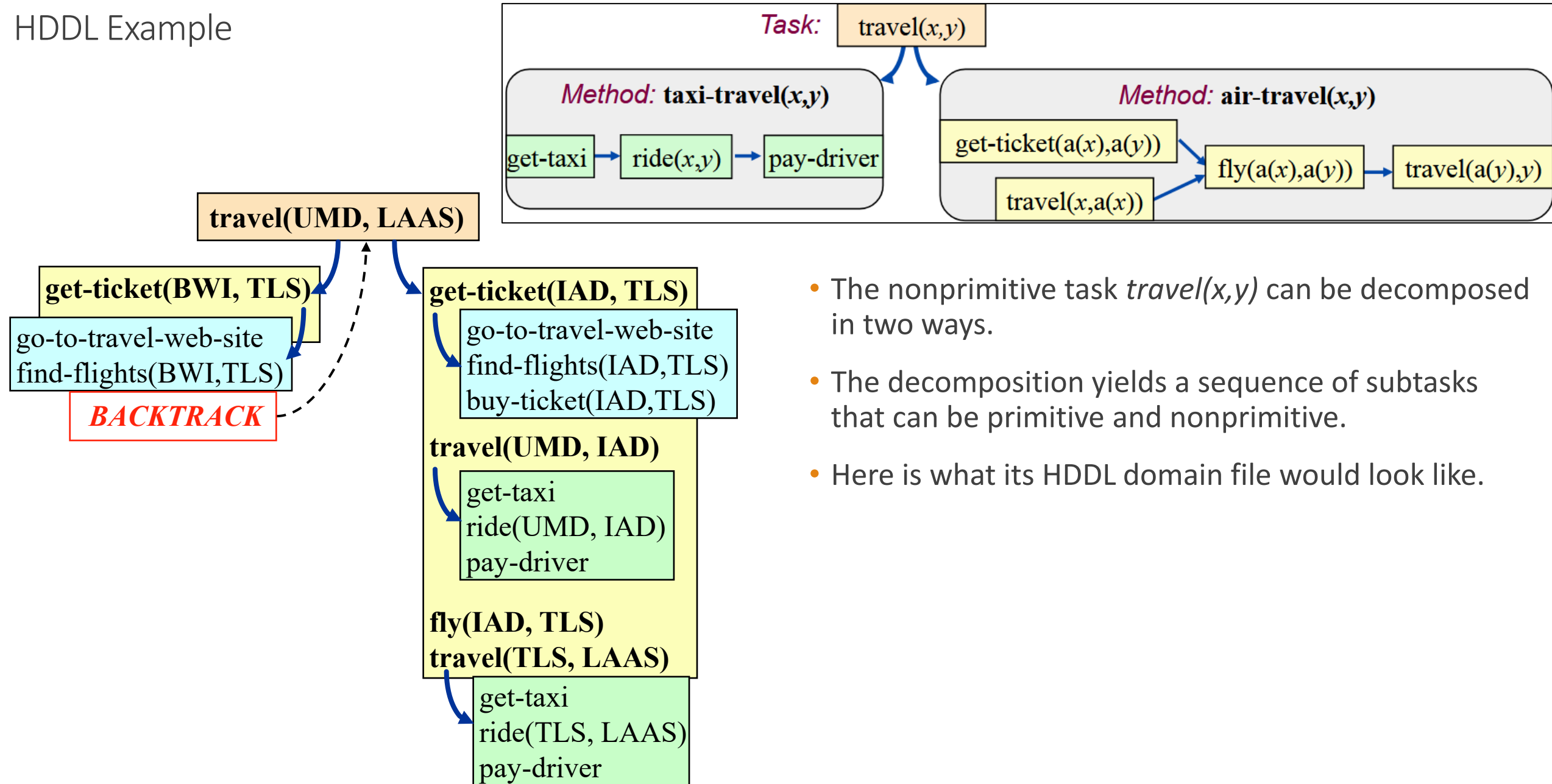


- The nonprimitive task $travel(x,y)$ can be decomposed in two ways.
- The decomposition yields a sequence of subtasks that can be primitive and nonprimitive.

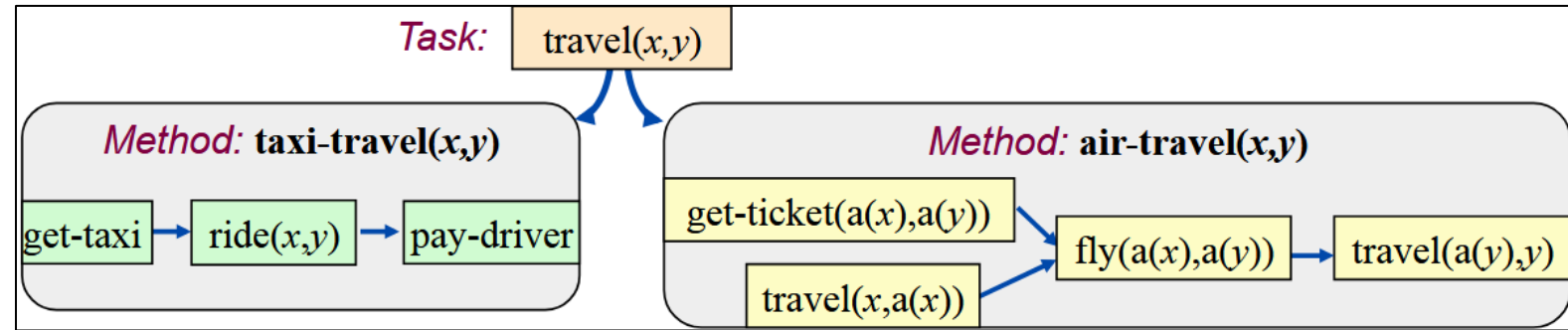
- Used in real-world applications as it can reduce search work significantly (divide-and-conquer).
- Can use different techniques for each level. Can use library of preconstructed plans for suitable levels.
- Popular hierarchical planners: Noah, Nonlin, O-Plan, SIPE, SIPE-2, SHOP, SHOP2
- Operations planning: Many deployed systems organize work as high-level activities that are refined into timed, executable steps under constraints (power, time, safety, communications).
- [NASA Mars 2020 \(Perseverance\)](#) activity planning & scheduling (JPL). describes on-mission activity scheduling that adapts when execution does not match the nominal timeline. Replans at the level of activities, not only primitive commands. Connection to HTN:
 - The idea matches HTN: **abstract tasks** → **structured decomposition** → **primitives**, even when the implementation is scheduling + constraints.

HTN vs classical PDDL

- Classical PDDL defines model dynamics as **primitive** actions; search explores sequences of ground actions achieving a goal formula.
- HTN adds **methods** that define legal **decompositions** of **compound (high-level)** tasks; search also branches over which method to use and may backtrack when a branch fails.
- Advantage of HTN vs flat PDDL: methods encode expert “how-to” structure, so the planner can do less search in the full primitive space by planning at high level first (choosing decompositions), then handling each subtask (refining it further or planning primitives) in smaller subproblems.
- Disadvantage: you must design and maintain those methods.
- HTN is a configurable planner: Same general engine + PDDL-like primitives + domain-specific method libraries. Sits between fully domain-independent classical planning and domain-specific implementations.
- Hierarchical domain definition language (HDDL) extends PDDL for hierarchical planning development.. HDDL is used in planning competitions (see [link](#)).
- See [link](#) for those interested in HDDL syntax. Also see [link](#) here for HDDL examples.
- Open-source planner that supports HDDL [link](#).



- The nonprimitive task $travel(x,y)$ can be decomposed in two ways.
- The decomposition yields a sequence of subtasks that can be primitive and nonprimitive.
- Here is what its HDDL domain file would look like.



```

1 (define (domain travel-htn)
2   (:requirements :negative-preconditions :typing :hierarchy)
3   (:types
4     airport - place
5     place - object
6   )
7   (:predicates
8     (use-taxi ?from - place ?to - place)
9     (use-air ?from - place ?to - place)
10  )
11
12  (:task travel
13    :parameters (?x - place ?y - place)
14  )
15  (:task get-ticket
16    :parameters (?a1 - airport ?a2 - airport)
17  )
18

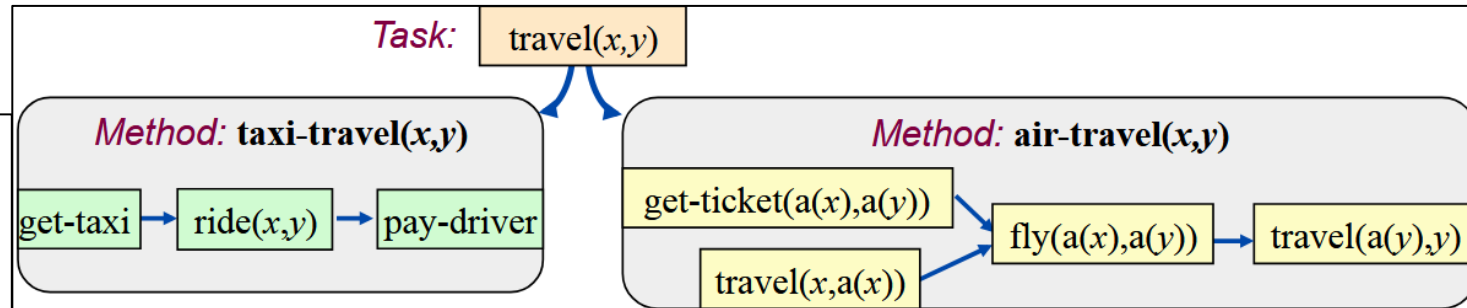
```

Notice
task
defined

```

19 ▾ (:method taxi_travel
20     :parameters (?x - place ?y - place)
21     :task (travel ?x ?y)
22     :precondition (use-taxi ?x ?y)
23 ▾   :subtasks (and
24       (t1 (get-taxi))
25       (t2 (ride ?x ?y))
26       (t3 (pay-driver))
27     )
28 ▾   :ordering (and
29     (< t1 t2)
30     (< t2 t3)
31   )
32 )
33 ▾ (:method get_ticket_web
34     :parameters (?a1 - airport ?a2 - airport)
35     :task (get-ticket ?a1 ?a2)
36 ▾   :subtasks (and
37     (g1 (go-to-travel-web-site))
38     (g2 (find-flights ?a1 ?a2))
39     (g3 (buy-ticket ?a1 ?a2))
40   )
41 ▾   :ordering (and
42     (< g1 g2)
43     (< g2 g3)
44   )
45 )

```



```

46 ▾ (:method air_travel
47     :parameters (?x - place ?y - place ?a1 - airport ?a2 - airport)
48     :task (travel ?x ?y)
49     :precondition (use-air ?x ?y)
50 ▾   :subtasks (and
51     (a0 (get-ticket ?a1 ?a2))
52     (a1 (travel ?x ?a1))
53     (a2 (fly ?a1 ?a2))
54     (a3 (travel ?a2 ?y))
55   )
56 ▾   :ordering (and
57     (< a0 a1)
58     (< a1 a2)
59     (< a2 a3)
60   )
61 )

```

Notice ordering defined

Primitive actions

```
63 ▾ (:action get-taxi
64     :parameters ()
65     :precondition (and)
66     :effect (and)
67 )
68 ▾ (:action ride
69     :parameters (?from - place ?to - place)
70     :precondition (and)
71     :effect (and)
72 )
73 ▾ (:action pay-driver
74     :parameters ()
75     :precondition (and)
76     :effect (and)
77 )
78 ▾ (:action go-to-travel-web-site
79     :parameters ()
80     :precondition (and)
81     :effect (and)
82 )
83 ▾ (:action find-flights
84     :parameters (?a1 - airport ?a2 - airport)
85     :precondition (and)
86     :effect (and)
87 )
88 ▾ (:action buy-ticket
89     :parameters (?a1 - airport ?a2 - airport)
90     :precondition (and)
91     :effect (and)
92 )
93 ▾ (:action fly
94     :parameters (?a1 - airport ?a2 - airport)
95     :precondition (and)
96     :effect (and)
```

Why use Hierarchical Planning?

- Main advantage is hierarchy. Break a complex mission into layered subtasks instead of one flat search problem. Many real-world problems naturally decompose.
- Scales better in practice. Decomposition and domain knowledge reduce effective search complexity on large tasks.
- Reusable task knowledge. Methods and tasks capture reusable procedures (e.g., deliver object, inspect area) across many scenarios.
- Fits robot architectures and development:
 - high-level symbolic planner decides *what to do*.
 - low-level controllers (like PID control) decide *how to execute safely*.
 - Task logic is decoupled from motion and control loops, making systems easier to maintain and debug.
- Used for workflow planning in logistics, manufacturing, game AI, and business processes.
- Classical search (BFS, DFS, A*) remains foundational, but hierarchical task planning is often the practical choice for robotic mission-level autonomy.

Some Interesting Links

- [HDDL paper](#)
- [HATP: An HTN Planner for Robotics](#)
- PlanSys2 docs: [ROS2 Planning System Documentation](#)
- ROSPlan docs (PDDL planning in ROS): [ROSPlan Overview](#)
- [Unreal Engine HTN Plugin](#)